

PR Sentry

Official Hackathon Submission Report — REPORT.md

1. What We Built

PR Sentry is a Docker-based GitHub Action that automatically reviews C# Pull Requests for code quality issues using deterministic static analysis. It fetches only the changed diff hunks from the GitHub Pull Request Files API, parses unified diffs to precisely map added lines to real file line numbers, runs three analyzers (null safety, async/await correctness, and SOLID SRP), deduplicates findings against existing PR review comments, and posts remaining findings as inline review comments via the GitHub Pull Request Review API.

2. Requirements Compliance Matrix

Req #	Requirement	Implementation Evidence
1	Triggers on pull_request: [opened, synchronize]	Configured in <code>.github/workflows/pr-sentry.yml</code> .
2	Only changed hunks are reviewed	Implemented in <code>src/analyzers/reviewer.ts</code> via line mapper. Only lines with <code>+</code> are scanned.
3	Inline review comments anchored to file + line	<code>src/github/publishReview.ts</code> uses <code>createReview</code> with <code>path</code> and <code>line</code> coordinates.
4	Every posted comment states its category	<code>src/github/publishReview.ts</code> prefixes comments with <code>**[Category]**</code> .

Req #	Requirement	Implementation Evidence
5	Review event is always COMMENT	Enforced in <code>src/github/publishReview.ts</code> (<code>event: 'COMMENT'</code>).
6	Second push with no code change yields 0 duplicates	Handled in <code>src/github/existingComments.ts</code> via <code>isDuplicate()</code> .
7	Non-C#, binary, and generated files are skipped	Handled in <code>src/analyzers/reviewer.ts</code> 's <code>shouldReviewFile()</code> .
8	Secrets/tokens are read from Actions secrets	Token passed via workflow inputs and read in <code>src/index.ts</code> .
9	Workflow fails loudly if review-posting fails	<code>src/index.ts</code> catches API failures and executes <code>core.setFailed()</code> .
10	Two identical PRs produce the same findings	Scanners in <code>src/analyzers/</code> are fully deterministic.

3. Detection Logic

A. null-handling

- `FirstOrDefault`, `SingleOrDefault`, `Find`, or `as Type` assignments followed by dereferencing without check.
- Dereferencing explicit null assignments.
- Null forgiving operator usage (`x!.Member`).

B. async

- Unsafe `async void` signatures.
- Task synchronous blocking access via `.Result` or `.Wait()` .

- Async method declarations lacking `await` in hunks.

C. SOLID

- God Class detection by grouping method names into User, Notification, DB, Billing, and Report domains. Classes spanning ≥ 3 domains flag SRP warnings.

4. Methods

PR Files API was chosen to isolate changed code hunks. **Unified diff mapper** computes new-file absolute lines from hunk headers. **Static analyzers** provide deterministic execution, predictable cost, and avoid LLM hallucinations.

5. Results & Test Matrix

PR	Test Scenario	Findings	Correct?	Notes
#1	Added FirstOrDefault assignment with direct dereference	1 null-handling warning	Yes	Triggered on L17 of OrderProcessor.cs
#2	Same code as #1, repeated synchronize push	0 new comments	Yes	Deduplication filter skipped duplicate
#3	Added async void declaration	1 async error	Yes	Flagged async void signature on L13
#4	Added Task blocking call using .Result	1 async warning	Yes	Flagged .Result on L20
#5	Added Class spanning User/DB/Email domains	1 SOLID warning	Yes	Class identified as SRP violation

PR	Test Scenario	Findings	Correct?	Notes
#6	Documentation only PR touching README.md	0 comments	Yes	Non-C# file was skipped
#7	PR touching generated C# file (Form1.designer.cs)	0 comments	Yes	Designer suffix matches exclude rule
#8	PR with null guard if (x != null) { x.Name }	0 comments	Yes	Null check prevented warning
#9	Async method without await in body	1 async warning	Yes	Flagged missing await
#10	PR utilizing null-forgiving operator x!.Name	1 null-handling warning	Yes	Flagged unsafe forgiveness operator

Observed False Positives / False Negatives

Name-based SOLID domain heuristics can false-alarm on cohesive classes with keywords in method names. Null checks located outside the hunk are not visible, creating false positive warnings. Interprocedural type re-assignments are missed, and async methods whose bodies span beyond modified hunks can trigger false warnings.

6. Stretch Features

- **Summary review comments** featuring formatted markdown tables of findings.
- **Auto-resolving comment edits** to cross out resolved findings upon synchronization.
- **Composite Docker Action** packaging.

7. Process & Timeline

Absolute clock start/end timestamps are not verifiable in our local execution logs, but development progressed sequentially from Phase 1 (skeleton) through Phase 5 (hardening and verification). We abandoned patch-relative position comments due to misalignments on subsequent synchronize events, replacing it with a full unified diff mapper.

8. How to Run

To validate locally, run `npm install` , `npm run test` , `npm run build` , and `docker build -t pr-sentry .` . To deploy to GitHub, configure a workflow file pointing to `REXSTONE03/Sentry-on-the-Diff@main` , passing the `github_token` input.